SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

The strategic solution for CloThinks is a comprehensive AI-powered decision intelligence platform designed to modernize SME fashion retail by replacing intuition-based habits with data-backed strategies. Centered around Z.AI's General Language Model (GLM), the system acts as an intelligent partner that processes both structured inventory data and unstructured data (fashion trends, customer preferences) to provide real-time tracking, demand prediction, and trend-aware optimization.
The solution specifically targets the challenge of deadstock through its "AI Fashion Stylist" function, which identifies slow-moving items and generates creative outfit combinations paired with trending pieces to turn unsold inventory into profitable bundles. By utilizing a multi-step agentic prompting strategy, CloThinks provides clear, human-understandable explanations for its recommendations, allowing store owners to confidently make decisions that increase revenue while simultaneously reducing textile waste and promoting a more sustainable fashion ecosystem.

1.1. Purpose:

This System Analysis Document defines the ***technical scope, architecture, and design decisions*** *behind CloThinks.*
Key elements covered include:

1. ***Architecture****:*
   - *[e.g. It's a cloud native platform. The services are separated into Microservices. Deployed on AWS and these microservices work independently.]*
   - *CloThinks is a* ***cloud-native microservices platform****, where services such as inventory, recommendation, and analytics operate independently for scalability and flexibility.*
2. ***Data Flows from:***
   - *User interfaces (web dashboards)*
   - *Backend APIs*
   - *Stored in databases (inventory & sales)*
   - *Processed by AI models*
   - *Returned as insights and recommendations*

3. ***Model Process:***
   - *[e.g. The model shows the end-to-end workflow of the core features: placing an order, assigning a rider for that specific order and confirming the delivery.]*

The system workflow includes:

1. *Capturing inventory and sales data*
2. *Processing through AI models*
3. *Generating predictions and recommendations*
4. *Delivering actionable insights to users*

© UMHackathon 2026

4. Role of Reference:

*This document serves as a **technical reference** for developers, designers, and QA teams to understand system architecture and implementation.*

*1.2. **Background**:*

Many clothing SMEs rely on intuition instead of data when making business decisions. Their data is often scattered across spreadsheets or manual records, making it difficult to gain insights or respond to fast-changing fashion trends. This leads to issues such as overstocking, missed trends, increased costs, and reduced profitability. Additionally, most retailers lack tools to analyze both sales data and external factors like customer preferences and trends, limiting effective decision-making.

1. *Previous Version:*
   - *Existing systems are mainly manual or basic POS platforms that only record sales and inventory. They do not provide analytics, forecasting, or integration with trend data. Store owners rely heavily on experience rather than data-driven insights.*

2. *Changes in Major Architectural Components:*
   - *CloThinks shifts to a **cloud-based AI-driven microservices architecture**, integrating Inventory, Analytics, and Recommendation services. It includes a **LLM (Z.AI GLM) as a service layer** for intelligent data processing and decision-making.*

3. *New capabilities that are only introduced in this system:*
   - *AI demand forecasting based on past sales and trends*
   - *Smart outfit recommendations for unsold stock*
   - *Integration of structured and unstructured data such as sales and fashion trends*
   - *Real-time inventory tracking*
   - *Data-driven decision support with AI recommendation*

1.3. Target Stakeholder

© UMHackathon 2026

Stakeholders	***Roles***	***Expectations***
SME fashion retailers	*Primary business owners and strategic decision-makers.*	*To improve overall profitability by reducing waste from unsold stock and increasing sales through AI-enhanced customer engagement.*
Store/ Inventory managers	*Daily operational leads responsible for stock tracking and shelf management.*	*Real-time visibility into inventory levels and accurate demand forecasting to prevent both overstocking and stockouts.*
Production staff	*Teams responsible for manufacturing or procuring garments.*	*Data-driven insights into fashion trends and demand to optimize production cycles and minimize material waste.*
Online customers	*The end-users who browse and purchase products.*	*A personalized shopping experience with automated styling suggestions that help them visualize outfit combinations.*
Small online fashion sellers	*Micro-entrepreneurs managing inventory across social media or small e-commerce platforms.*	*Accessible, easy-to-use tools that automate inventory tracking and content creation (outfit styling) to save time and resources.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. *System Architecture & Design*

2.1. *High Level Architecture*

2.1.1. Overview:

Type	Details
System	*Website*
Architecture	*Cloud-Server with Microservices*

*CloThinks follows a **client-server architecture**:*

- ***Frontend:*** *Admin dashboard (store owners)*
- ***Backend API Layer:*** *Handles requests*
- ***Microservices:***
  - *Inventory Service*
  - *Recommendation Service*
  - *Analytics Service*
- ***Database:*** *Stores inventory & sales data*
- ***LLM Service (Z.AI GLM):*** *AI reasoning & recommendations*

2. LLM as Service Layer

CloThinks integrates GLM as a core service layer, not just a simple AI module:

Prompt Construction

Prompts include:

- Inventory data (unsold items)
- Sales trends
- Customer preferences

Context Window

- Top-selling items
- Deadstock items
- Current fashion trends

Response Handling

- GLM outputs structured recommendations
- Backend parses and converts into:
    - Outfit suggestions
    - Stock actions

Token Optimization

- Inputs are chunked
- Repeated queries are cached
- Prompt size is controlled

3. Dependency Diagram

- Frontend (Web Dashboard) sends user requests to Backend API
- Backend retrieves inventory and sales data from Database
- Backend constructs prompt (inventory + trends + user input)
- Prompt sent to Z.AI GLM API
- GLM processes and returns structured recommendations
- Backend parses response and sends results to Frontend
- Token limits enforced by truncating inputs and caching repeated queries

2.2.2. Sequence Diagram

- *User selects items or requests recommendation*
- *Frontend sends request to Backend*
- *Backend fetches relevant data from Database*
- *Backend sends processed data to GLM API*
- *GLM generates recommendation*
- *Backend processes response*
- *Frontend displays results to user*

2.2. Technological Stack

2.2.1. Frontend: React (Web Dashboard)
2.2.2. Backend: FastAPI – RESTful API for the microservices
2.2.3. Database: PostgreSQL – Relational Database for orders, users
2.2.4. Cloud/Deployment: AWS (EC2, RDS, S3)

2.2.5. AI Model: Z.AI GLM

© UMHackathon 2026

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. Data Flow Diagram (DFD)

a. Illustrate the data movement across the entire system that includes external entities (users), processes, external service and databases

b. Show how proper laws are met (e.g. Black Hole)

2.3.2. Normalized Database Schema

a. Show the logical data model (ERD) with Database designed to 3NF form to ensure data integrity and relationships. [e.g. order items is a junction table that is resolving the M: N relationship between orders and menu items.

3. Functional Requirements & Scope

3.1. Minimum Viable Product:

3.1.1. Core Features

#	***Features***	***Description***
1	*Smart Inventory Intelligence*	*The system tracks stock in real time and analyzes sales patterns to identify overstocked or slow-moving items.*
2	*AI Outfit Matching Recommendation*	*It functions as an AI fashion stylist, generating outfit combinations by pairing slow-moving items with bestselling pieces to turn unsold stock into sellable looks.*
3	*Demand Prediction*	*The platform analyzes sales trends and unstructured fashion data to predict future demand , suggesting proactive solutions like restocking or bundling.*
4	*Conversational Natural Language Interface*	*Users can interact with the system using simple English prompts to retrieve suggestions and insights , making complex data interpretation accessible.*

© UMHackathon 2026

3.2. *Non-Functional Requirements (NFRs)*

Quality	***Requirements***	***Implementation***
Performance & Scalability	*The system must handle high volumes of text data while maintaining context-aware reasoning.*	*Implements a 128,000 token limit per session with a smart chunking and summarization pipeline for oversized inputs.*
Reliability & Accuracy	*The system must prevent erratic suggestions and handle model failures without disrupting business logic.*	*Uses a Self-Correction Layer for automatic retries and defaults to a "Simplified Insight" fallback for human review.*
Transparency & Trust	*Recommendations must be understandable to store owners to ensure data-driven confidence.*	*The Z.AI GLM generates natural language explanations for every strategy, such as justifying specific outfit pairings.*
Security & Scope Control	*The system must prevent unauthorized or automated changes to business operations.*	*Restricts automated execution; all AI-generated recommendations and markdowns require manual human validation.*
Cost Efficiency	*Operational costs must be minimized to keep the service viable for small SMEs.*	*Caches repeated queries for 1 hour to reduce redundant inference by an estimated 40%.*

3.3. *Out of Scope / Future Enhancements*

The system must handle high volumes of text data while maintaining context-aware reasoning.

3.4. *Do identify the high-level features that the team planned for "Grand Vision" but not part of this UMHackathon 2026. [e.g. below]*

a. ***Advanced Image-Based Recognition****: The initial version is limited to text-based interaction and structured data. Future versions would include visual AI to recognize clothing styles directly from photos.*
b. ***Live Backend Integration (POS/ERP)****: Currently, data must be uploaded manually. The grand vision includes seamless, real-time integration with live Point-of-Sale (POS) and Enterprise Resource Planning (ERP) systems.*

© UMHackathon 2026

c. ***Automated Markdown Execution***: *Currently, the system only provides recommendations that require human approval. Future updates would allow the AI to automatically adjust prices or initiate discounts based on demand signals.*
d. ***Multi-User Simultaneous Input Support***: *The current scope is limited to single-user inputs at a time to ensure system stability. Future scaling will support concurrent collaborative sessions for larger teams.*
e. ***Real-Time Fashion Trend Scraping***: *While the system evaluates unstructured trends , a future enhancement would involve direct, automated scraping of global social media and trend reports for even faster "second-by-second" market responses.*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. Grayscale Rollout & A/B Testing:

Clothinks will deploy new features such as AI outfit recommendation, inventory prediction, demand forecasting using a controlled rollout approach:

✔ Phase 1: Internal Testing (0%)

- *Features tested by developers + internal admin panel*
- *Validate:*
  - *Inventory logic accuracy*
  - *Recommendation quality*
  - *API response stability*

✔ Phase 2: Grayscale Release (5% Users)

- *New feature (eg. AI outfit suggestion engine) released to **5%** of SME stores*
- *Example:*
  - *5 out of 100 clothing stores get AI styling suggestions*
- *Monitor:*
  - *Stock prediction accuracy*
  - *User engagement (clicks on suggestions)*
  - *Error rates (API failures, latency)*

✔ Phase 3: Expansion (25% → 50% → 100%)

- *If system stable:*
  - *Increase rollout gradually (25%, 50%, 100%)*
- *Stop expansion if:*
  - *Inventory mismatch increases*
  - *API cost spikes abnormally*

© UMHackathon 2026

- *Recommendation quality drops*

A/B Testing Strategy

Variant	Description
A	Rule-based outfit matching (colour + category logic)
B	LLM-based GLM AI fashion recommendation

4.1.2. Emergency Rollback (ER) & Golden Release:

If system detects:

- Inventory mismatch > 5%
- API failure rate > 2%
- GLM API timeout spikes
- Recommendation system producing incorrect matches

<u>Action:</u>

- Trigger Emergency Rollback (ER) Pipeline
- Immediately revert to Golden Release Version

<u>Golden Release:</u>

- Stable version of Clothinks system
- Includes:
  - Basic inventory tracking
  - Simple rule-based styling
  - No experimental AI models

<u>Recovery Flow:</u>

1. Detect anomaly via monitoring system
2. Auto-disable new AI module
3. Redirect traffic to stable version
4. Notify admin dashboard + alert SMS/email

4.1.3. Priority Matrix:

4.1.3.1.

© UMHackathon 2026

Priority	*Condition*	*Action*
P1 Critical	*Inventory mismatch > 5% OR payment/order failure > 1%*	*Immediate Emergency Rollback + disable AI recommendation*
P2 High	*API latency > 3 seconds OR GLM error rate > 10%*	*Switch to fallback model (rule-based system)*
P3 Medium	*Stock prediction accuracy drops < 80%*	*Trigger model retraining + log analysis*
P4 Low	*UI bugs, minor layout issues*	*Fix in next sprint update*

*4.2. **Monitoring**:*

4.2.1. Clothinks uses a real-time monitoring dashboard for SMEs and admin systems.

Inventory System:

- *Stock in/out accuracy*
- *Overstock vs understock rate*
- *Stock aging*

AI Recommendation System:

- *Recommendation click-through rate*
- *Conversion rate*
- *User engagement time*
-

System Health

- *API latency*
- *Error rate per service*
- *Server CPU & memory usage*

*4.3. **Assumptions:***

© UMHackathon 2026

Clothinks system is built based on these assumptions:

User Environment

- Users (SME owners) have stable internet access
- Mobile and desktop devices are available for dashboard use

Business Environment

- Shops manually update inventory or use barcode system
- Each store has at least one admin user managing stock

AI Behavior

- GLM API provides stable structured responses
- Fashion dataset used for training is representative of Malaysian market trends

Operational Assumptions

- Inventory data is updated daily or real-time
- Users follow recommended AI suggestions partially

4.4. ***External Dependencies:*** *Do list all the external tools that are applied to your system to function. [e.g. below]*

Tools	*Purpose*	*Risks*
Z Ai GLM API	*Core AI engine for fashion recommendation demand prediction*	*High risk: rate limits, downtime*
Cloud Database (AWS)	*Store inventory, user data, transaction logs*	*Medium: downtime or sync delay*
Notification Service (SMS/WhatsApp API)	*Alert SMEs about stock issues*	*Medium: delivery failure*
Analytics Engine (Google Analytics)	*Track user behavior & system performance*	*Low to medium: data delay*

© UMHackathon 2026

5. *Project Management & Team Contributions*

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. ***Project Timeline:*** *Show the timeline of development and justify accordingly [e.g. Day 1-2, Project Planning etc.]*

5.2. ***Team Members Role:*** *Highlight the role of team members [e.g. Person A – Role Backend Lead – Fast API Microservices & Schema Design]*

Day	***Date***	***Task***	***Person of responsibility***
1 -2	*16/4/2026-* *17/4/2026*	*Domain Reveal and Research* *· Outline problem statement* *Project Design Planning and Requirement Analysis* *·Identify scenario* *·Define problem description and propose solution (inventory inefficiency, deadstock, AI styling)* *·Review PRD scope, constraints, and success metrics* *·Assign roles* *·Define system architecture (AI pipeline, dashboard, data flow)*	*ALL* ✅

© UMHackathon 2026

3	*18/4/2026*	*Data design and Preparation* *·Design inventory dataset (stock levels, sales history, categories)* *·Simulate or clean sample SME retail data* *·Define input/output format for AI model (GLM prompts)* *Complete Document* *·Complete Product Requirement Document*	*Angeline* ✅
4-5	*19/4/2026-* *20/4/2026*	*Core Backend Development* *·Implement:* *- Inventory tracking logic* *- Slow-moving item detection* *- Basic demand prediction rules (heuristic or simple model)*	*Sharon* ✅
6-7	*21/4/2026-* *22/4/2026*	*AI Integration (Z.AI GLM)* *·Implement prompt pipeline:* *- Identify slow-moving items* *- Generate outfit combinations* *- Provide explanations* *·Add fallback mechanism (retry and simplified insight)*	*Jolin* ✅

© UMHackathon 2026

8	*23/4/2026*	*Frontend Development (UI/UX)* *·Build dashboard:* *- Inventory overview* *- AI recommendation display* *·Implement conversational interface (prompt input)* *Complete Document* *· Complete System Analysis Document*	*Angeline,* *Sharon* ✅
9	*24/4/2026*	*Integration and Testing* *· Connect frontend, backend, and AI* *· Test:* *- Prompt-to-insight flow* *- Outfit generation flow* *- Demand prediction outputs* *·Fix bugs and inconsistencies* *Complete Document* *·Complete Quality Assurance Testing Document*	*Jolin* ✅

© UMHackathon 2026

Email: umhackathon@um.edu.my

10	*25/4/2026*	*Final Optimization and Presentation Preparation* *· Optimize response time* *· Prepare demo scenarios:* *- "Sell leftover items"* *- "Predict demand"* *·Finalize slides and pitching narrative* *Demonstration* *·Record pitching video with prototype demonstration*	*ALL* ✅

Recommendations:

1. Performance & Scalability

- ***Redis Caching for Session Data***: *Similar to your example of caching menu data, the PRD explicitly mentions caching repeated queries for 1 hour to reduce redundant inference by ~40%. Implementing a* ***Redis cache*** *would be the ideal way to manage these high-density prompts and "style signals".*
- ***Asynchronous Processing for Data Pipelines***: *Since the system uses a sophisticated preprocessing pipeline for "oversized text" (breaking down data into segments and recombining them), using a task queue (like Celery or RabbitMQ) would prevent the natural language interface from hanging while the AI "thinks".*
- ***Vector Database for Trend Analysis***: *To handle "unstructured fashion trends" and "customer reviews" efficiently, a vector database (like Pinecone or Milvus) could store style embeddings, allowing the Z.AI GLM to retrieve relevant "style signals" faster than scanning raw text.*

2. Reliability & Accuracy

- ***Circuit Breaker Pattern***: *To implement the "Fallback & Failure Behavior" described in the PRD, a circuit breaker can monitor the Z.AI GLM. If the model fails consistently, the system automatically "trips" to the* ***Simplified Insight*** *fallback to protect the user experience.*
- ***Automated Validation Layer (Self-Correction)***: *The "Self-Correction Layer" mentioned in the document can be implemented as a programmatic check that compares AI-generated outfit pairings against a hard-coded "Aesthetic Constraint" database to prevent "hallucinated" or incompatible combinations.*

© UMHackathon 2026

3. Security

- ***Rate Limiting****: Since the system currently does not support "Multi-user inputs simultaneously," implementing rate limiting at the API level is crucial to ensure one user's heavy data upload doesn't crash the session for others.*
- ***Audit Logging for Human Validation****: Because the PRD requires "human validation before execution" for markdowns and recommendations, the system should include a robust logging mechanism that tracks which manager approved which AI suggestion, ensuring accountability.*

© UMHackathon 2026

Email: umhackathon@um.edu.my